

Taller de aula: el formulario le habla a una pieza que no ves

Programación 3 - Tecnicatura Universitaria en Programación - UTN FRM.

UNIDAD 4 - EL DESPLIEGUE.

Integrantes: Avila Lucas, Pagano Amanda, Liendo Mateo.

Modalidad	En grupos de dos o tres, en el aula. Se entrega antes de irse : lo que no se termina en la mesa no se termina.
Requisitos	No hace falta tener el VPS andando. Tampoco hace falta internet ni entrar a Easypanel. Solo el apunte del capitulo 4 a mano.
Tiempo	70 minutos. Partes 1 a 5, mas la Parte 6 que se lleva al proyecto.
Puntaje	100 puntos: 1 (20) + 2 (15) + 3 (20) + 4 (15) + 5 (20) + 6 (10).
Que se evalua	Criterio de configuracion y de diagnostico: saber a que pieza le habla cada campo del formulario, y poder ir del sintoma que se ve al mecanismo que lo produjo, que casi nunca esta en el mismo lugar.

Parte 1 - El formulario y la pieza que configura por debajo

20 puntos

Esta es la configuración completa de los dos servicios del proyecto, tal como queda cargada en Easypanel. Seis campos, dos servicios, y cada campo le habla a un componente distinto de los tres que el panel coordina.

```

--- Servicio: api -----
1 Source > Build path      /
2 Environment              ORIGENES_PERMITIDOS=https://calculadora.tudominio.com
3 Domains & Proxy > Dominio api.tudominio.com
4 Domains & Proxy > Puerto 8000
5 Domains & Proxy > HTTPS  activado

--- Servicio: web -----
6 Environment              API_URL=https://api.tudominio.com

```

1.1 Completa la traducción

12 puntos (2 por fila)

Una fila por campo. En "A quien le habla" va uno de estos tres: Docker, Traefik o el proceso de la aplicación. En "Cuando se lee" importa distinguir si el valor actúa al construir, al arrancar el contenedor o en cada petición.

#	A quien le habla	Que configura por debajo	Si esta mal, que se ve	Cuando se lee ese valor
1	Docker	El contexto de construcción: el directorio que se envía al motor de Docker	La construcción falla (no encuentra el Dockerfile/directorio)	Al construir la imagen
2	El proceso de la app (API/FastAPI)	La lista de orígenes autorizados que la API declara en la cabecera CORS	Error "CORS policy" en la consola, aunque el backend conteste 200	En cada petición (pero solo toma el valor nuevo tras redeploy)
3	Traefik	Un enrutador (router): la regla Host(api.tudominio.com)	404 — ningún enrutador coincidió	Al arrancar el contenedor (Traefik lo descubre sólo)
4	Traefik	El servicio: el destino interno (contenedor + puerto) al que se entrega la petición	502 — el enrutador coincidió pero el servicio no contestó	Al arrancar / en cada petición enrutada
5	Traefik + Let's Encrypt	La emisión del certificado vía ACME, desafío HTTP-01 (usa el puerto 80)	Sitio sin HTTPS, certificado "pendiente"	Al desplegar y luego en cada renovación (cada ~90 días)
6	Docker (entrypoint del contenedor web)	Genera config.js, que el frontend usa para saber a qué API hablar	Mixed Content (si es http) o ERR_CONNECTION_REFUSED a 127.0.0.1 (si no se generó)	Al arrancar el contenedor

1.2 El puerto que no abre ningún puerto

4 puntos

Un integrante dice: "si puse 8000 en Domains & Proxy, entonces el 8000 del VPS quedó abierto, y eso contradice todo lo que hicimos con el firewall en la Clase 2". Decime si tiene razón, qué resultado daría un nmap desde afuera contra ese puerto, y por qué.

No tiene razón. Un nmap desde afuera contra el puerto 8000 daría cerrado (o "filtered", como en el nmap de la Parte 5 del taller para el puerto 80 bloqueado mismo tipo de resultado).

Por qué: el campo Puerto en Domains & Proxy es el puerto interno del contenedor, no un puerto que se abre en el VPS hacia afuera. Le está diciendo a Traefik "cuando llegue algo para api.tudominio.com, mandárselo a este contenedor, al 8000" es una instrucción de enrutamiento interno, no una apertura de firewall. Los únicos puertos que están realmente abiertos al mundo son el 80 y el 443, que son los que atiende Traefik. Todo lo demás vive en la red interna del proyecto y nadie de afuera puede tocarlo directo.

1.3 Los dos números que no son el mismo

4 puntos

El campo Puerto vale 8000 en el servicio api y 80 en el servicio web. Decime de donde sale cada uno de esos dos números -no "del apunte": que archivo o que proceso lo determina- y que error concreto produce copiar la configuración del backend al frontend cambiando solo el dominio.

Son un reflejo de qué programa corre dentro de cada contenedor. El 8000 aparece porque el backend está armado en FastAPI y corre sobre uvicorn, y uvicorn por default queda escuchando ahí. El 80 aparece porque el frontend no es Python, es HTML servido por nginx, y nginx escucha en el puerto web que es el 80. Entonces cuando se carga ese número en Domains & Proxy no se inventa nada ni "asigna" un puerto. Le estamos diciendo a Traefik en qué puerto interno tiene que buscar al proceso que realmente está corriendo ahí adentro. Por eso, si se copia la configuración del backend para armar la del frontend y solo se cambió el dominio, nos va a quedar el 8000 puesto en un contenedor donde el que escucha es nginx en el 80. Traefik hace bien su parte: encuentra el enrutador, ve que el dominio coincide, y va a tocarle la puerta al contenedor... pero en el puerto equivocado. Ahí no le contesta nadie, así que el resultado es un 502. No es que "se rompió el dominio" (eso sería 404), es que se rompió el último tramo, encontró a quién preguntarle pero le preguntó mal.

Parte 2 - La cadena de Traefik: donde se rompio

15 puntos

Traefik encadena cuatro conceptos y los atraviesa siempre en el mismo orden: llega una petición a un **punto de entrada**, se evalúan los **enrutadores** hasta encontrar uno cuya regla coincida, se aplican sus **middlewares** y se entrega al **servicio**. Cinco grupos reportan cinco síntomas distintos sobre esa misma cadena.

- A

El dominio del frontend devuelve 404, con una pagina de Traefik.
- B

El dominio del frontend devuelve 502 Bad Gateway.
- C

El dominio responde, pero el navegador avisa que el certificado no es valido para ese nombre.
- D

El sitio carga por HTTPS y la calculadora no calcula: la consola dice CORS policy y el log del backend muestra 200 OK.
- E

El dominio no responde nada: la conexion ni siquiera se establece.

2.1 Ubica cada síntoma en la cadena

8 puntos

Una fila por caso. En "Eslabón" escribí hasta donde llegó la petición antes de fallar. En "Que descarta" va lo que ese síntoma te permite dejar de sospechar: la mitad del valor de un diagnóstico es lo que elimina.

#	Eslabón donde se detuvo	Causa más probable	Que descarta el síntoma
A	404, página de Traefik. No llegó a encontrar un enrutador que matchee	El dominio no está bien configurado en Domains & Proxy del servicio (o directamente no existe ese servicio)	Descarta que el problema sea del contenedor: ni siquiera llegó a buscarlo
B	502 Bad Gateway Pasó el enrutador, se rompió en el servicio	El contenedor no responde en el puerto declarado (puerto mal puesto, o el proceso escucha en 127.0.0.1 en vez de 0.0.0.0)	Descarta problema de dominio/DNS: Traefik ya sabía a quién mandarle la petición
C	Certificado inválido para ese nombre Pasó el enrutador y el servicio, se rompió en la capa TLS/ACME (antes incluso del entryptpoint, en términos de la negociación)	El certificado se emitió para otro dominio, o hay un desacuerdo entre lo que dice el SNI y lo que dice el Host	Descarta que Traefik esté enrutando mal el tráfico HTTP en sí el routing funciona, lo que falla es la identidad del certificado
D	Carga por HTTPS, no calcula, consola dice CORS, backend loguea 200Llegó hasta el final de la cadena de Traefik sin problema — el corte está después, en el navegador	Falta el dominio del frontend en ORIGENES_PERMITIDOS, o hay un desajuste (con/sin www, http/https)	Descarta cualquier problema de Traefik, de puerto o de servidor: el servidor contestó bien
E	No responde nada, ni conecta. Ni siquiera llegó al punto de entrada	El DNS no resuelve hacia el VPS, o el firewall tiene cerrado el 80/443	Descarta absolutamente todo lo de Traefik/CORS/certificado: no hay ni conexión

2.2 Explicarlo en una frase de ingeniería

4 puntos

Completa: "Un 404 y un 502 no son dos versiones del mismo problema: el 404 dice que **Traefik ni siquiera encontró a quién entregarle la petición, el dominio no está asociado a ningún servicio**, y el 502 dice que **sí encontró a quién, pero cuando fue a golpearle la puerta, nadie contestó**. Por eso, frente a un 502, revisar el dominio es **perder el tiempo: el dominio ya demostró que funciona, porque si no funcionara directamente tendrías un 404.**"

2.3 El nombre que viaja dos veces

3 puntos

En una conexión HTTPS el nombre del sitio viaja dos veces. Nombra los dos lugares, deci cual de los dos va cifrado, y explica por que el orden entre ambos no podría ser el inverso.

Los dos lugares son:

- La extensión SNI, dentro del saludo TLS (el handshake inicial de la conexión).
- La cabecera Host, dentro de la petición HTTP.

El que va cifrado es el segundo, la cabecera Host. El SNI va en claro, sin cifrar.

Por qué el orden no podría ser al revés: para que el servidor pueda cifrar la conexión, primero tiene que elegir qué certificado presentarle al cliente y elegir el certificado correcto depende de saber qué dominio pidió el usuario. Pero el canal cifrado todavía no existe en ese momento, porque cifrar es justo lo que se está por negociar. Entonces hay una especie de huevo y gallina: si el nombre del dominio viajará solo cifrado (dentro del Host), el servidor necesitaría el canal cifrado para leerlo, pero necesita saber el nombre antes de poder armar ese canal cifrado. La solución es que el nombre se declare una primera vez en claro (SNI), al inicio del saludo, para que el servidor pueda elegir certificado; recién después, ya con el canal armado, el nombre se repite cifrado en el Host de la petición real. Si el orden fuera al revés, el servidor jamás podría empezar a cifrar, porque no sabría con qué certificado hacerlo.

Parte 3 - El orden del despliegue

20 puntos

Doce pasos, desordenados, para publicar el proyecto completo desde cero. **Diez se hacen en el panel de Easypanel; dos no**, y donde van esos dos es la mitad del ejercicio. Igual que con la receta de la clase pasada, el orden aca no es cuestion de estilo: hay posiciones que rompen la emision del certificado, posiciones que no rompen nada y te cuestan una hora buscando donde no es, y una que entrega algo que despues no se retira del todo.

- a) Cargar ORIGENES_PERMITIDOS=https://calculadora.tudominio.com en el servicio api
- b) Configurar Domains & Proxy del web: calculadora.tudominio.com, puerto 80, HTTPS activado
- c) Crear el registro comodin *.tudominio.com apuntando a la IP del VPS
- d) Desplegar el api y verificar https://api.tudominio.com/api/salud
- e) Instalar la aplicacion de GitHub dando acceso a los repositorios
- f) Crear el servicio api: origen GitHub, repositorio del backend, branch main, Build path /
- g) Abrir los puertos 80 y 443 en el firewall del VPS
- h) Configurar Domains & Proxy del api: api.tudominio.com, puerto 8000, HTTPS activado
- i) Crear el proyecto calculadora en Easypanel
- j) Cargar API_URL=https://api.tudominio.com en el servicio web
- k) Crear el servicio web: repositorio del frontend, branch main, Build path /
- l) Abrir el frontend con F12 y verificar que la pestana Red muestre OPTIONS 200 seguido de POST 200

3.1 Ordena los doce pasos

10 puntos

Escribí las doce letras en el orden correcto de ejecución.

1. c - Crear el registro comodín *.tudominio.com IP del VPS (prerrequisito de Clase 1)
2. g - Abrir puertos 80 y 443 en el firewall (prerrequisito de Clase 2)
3. i - Crear el proyecto calculadora en Easypanel
4. e - Instalar la app de GitHub dando acceso a los dos repos
5. f - Crear el servicio api (Source: GitHub, repo backend, branch main, Build path /)
6. a - Cargar ORIGENES_PERMITIDOS en api
7. h - Domains & Proxy del api (dominio, puerto 8000, HTTPS)
8. d - Desplegar el api y verificar /api/salud
9. k - Crear el servicio web (repo frontend, branch main, Build path /)
10. j - Cargar API_URL en web
11. b - Domains & Proxy del web (dominio, puerto 80, HTTPS)
12. l - Abrir el frontend con F12 y verificar OPTIONS 200 + POST 200

3.2 Marca los tres puntos de no retorno

6 puntos (2 por punto)

Tres de esos pasos, puestos fuera de lugar, hacen daño de tres clases distintas: uno rompe en el acto y con un mensaje que no dice la verdad, uno no rompe nada visible y te cuesta una hora en cada intento, y uno entrega algo que después no se retira del todo. Identificarlos y decime, en cada caso, qué error concreto los convierte en trampa.

1. **g (firewall) fuera de lugar:** rompe la emisión del certificado en el acto. Si abris el 80/443 tarde, ACME no puede validar el dominio y el certificado queda "pendiente" sin que Easypanel te diga que el problema es el firewall
2. **a (ORIGENES_PERMITIDOS) fuera de lugar:** no rompe nada visible del lado del servidor: el backend sigue logueando 200 OK como si todo anduviera, y perdés el tiempo buscando ahí donde no está el problema, porque en realidad es CORS del lado del navegador
3. **e (GitHub, dar acceso a "All repositories"):** entrega algo que después no se retira del todo: aunque te des cuenta y lo corrijas, ya diste acceso de más durante ese tiempo

3.3 Los dos pasos que Easypanel no puede hacer por vos

4 puntos

Identifica los dos pasos que no se hacen en el panel. Para cada uno, explica por qué el panel no puede hacerlo y cómo se manifiesta su ausencia: los dos se manifiestan como un problema de Easypanel, y no lo son.

Los dos pasos que Easypanel no puede hacer:

- c (registro DNS, se hace en el proveedor de dominio, no en el panel) y
- g (firewall del VPS, es sistema operativo, no Easypanel). Los dos se disfrazan de "problema de Easypanel" el certificado queda pendiente o el dominio no responde, y no hay botón en el panel que lo arregle.

Parte 4 - Auditoria de una configuracion entregada

15 puntos

Un grupo entrego esta ficha como "configuracion lista para desplegar". Tiene **cinco problemas reales** -uno de ellos con dos defectos distintos en la misma linea- y **una linea que parece un problema y no lo es**.

```
--- Servicio: api -----
1 Repositorio    MatyAlts/calculadora-backend    Branch: main
2 Build path     /backend
3 Environment     ORIGENES_PERMITIDOS=http://calculadora.tudominio.com/
4 Domains        api.tudominio.com    Puerto: 8000    HTTPS: si

--- Servicio: web -----
5 Repositorio    MatyAlts/calculadora-frontend    Branch: main
6 Build path     /
7 Environment     API_URL=https://api.tudominio.com/api/calcular
8 Domains        calculadora.tudominio.com    Puerto: 8000    HTTPS: si

--- Notas del grupo -----
9 "cerramos el 80 en ufw despues de que salieron los certificados,
   total todo el trafico va por el 443"
10 "en produccion vamos a poner los dos dominios separados por coma,
    con www y sin www, porque son dos"
```

4.1 Los cinco problemas

10 puntos (2 por problema)

Para cada uno: número de línea, corrección concreta y una línea de justificación. Sin justificación no suma.

- Línea 2/3 – Problemas en el Build path, debería ser "/"
- Línea 3 – Utiliza el http en vez de https, además de eso, tiene "/" al final. El origen debe coincidir exactamente con https://calculadora.tudominio.me
- Línea 7 – La API_URL debe ser la URL base de la API, sin el path del endpoint. Deberia de quedar como https://api.tudominio.com
- Línea 8 – El frontend debe quedar expuesto internamente por el puerto 80, poner el puerto 8000 genera un error 502 si el contenedor escucha en 80
- Línea 9 – HTTP-01 de Let's Encrypt necesita el puerto 80, incluso para futuras renovaciones, si lo cierran esto impide la renovación

4.2 La línea que no es un problema, y una propuesta

5 puntos

Dos preguntas. (i)Cuál de las diez líneas parece un problema y no lo es, y por que. (ii) Otro integrante propone borrar directamente la variable de la línea 3, argumentando: "el apunte dice que CORS no protege la API, así que esa variable no sirve para nada y encima nos está rompiendo el despliegue". Decime si tiene razón en el hecho y por que igual la decisión es incorrecta.

(i)Línea que parece un problema pero no lo es: Línea 10 – Que haya dos dominios apuntando al mismo servicio no es un problema en sí mismo, porque ambos pueden asociarse al frontend.

(ii) Tiene razón en decir que CORS no protege la API, ya que es una política del cliente, no una defensa. Pero de allí a borrar esa línea, está equivocado porque el problema está en la configuración del despliegue, no es la línea en sí. Si quiere solucionarse ese problema es cambiar de <http://calculadora.tudominio.com/> a <https://calculadora.tudominio.com>

Parte 5 - Peritaje: el sitio que carga y no calcula

20 puntos

Un grupo pide ayuda: "el sitio abre con candado, la API abre con candado, y la calculadora no calcula. Cambiamos la variable tres veces y siempre da lo mismo". Un rato despues escriben de nuevo: "ah, y publicamos la documentacion en un tercer subdominio y ese quedo sin certificado hace 40 minutos". Esta es toda la evidencia que trajeron. Leela entera antes de escribir nada.

```
$ nslookup calculadora.tup2026.com    -> 148.230.94.12
$ nslookup api.tup2026.com            -> 148.230.94.12
$ nslookup docs.tup2026.com           -> 148.230.94.12

--- Consola del navegador, en https://www.calculadora.tup2026.com ---
Access to fetch at "https://api.tup2026.com/api/calcular" from origin
"https://www.calculadora.tup2026.com" has been blocked by CORS policy:
No "Access-Control-Allow-Origin" header is present on the requested
resource.

--- Pestana Red del navegador ---
OPTIONS /api/calcular 200
(no figura ningun POST)

--- Logs del servicio api (Easypanel) ---
INFO: "OPTIONS /api/calcular HTTP/1.1" 200 OK
INFO: "POST /api/calcular HTTP/1.1" 200 OK    <- hace 40 min, desde /docs

--- Panel del servicio api ---
Environment: ORIGENES_PERMITIDOS=https://calculadora.tup2026.com
Ultima edicion de variables: hace 6 minutos
Ultimo deploy:             hace 3 horas
Estado:                    running

--- Logs del servicio web ---
config.js generado con API_URL=https://api.tup2026.com

--- Desde la terminal de un alumno ---
$ curl -X POST https://api.tup2026.com/api/calcular \
  -H "Content-Type: application/json" \
  -d "{ a: 2, b: 3, operacion: suma }"
{"resultado":5.0}

--- Tercer servicio: docs.tup2026.com ---
Easypanel: certificado pendiente hace 40 minutos
$ nmap -p 80,443 148.230.94.12
80/tcp    filtered
443/tcp   open
```

5.1 Reconstruir la cadena de hechos

6 puntos

En seis líneas como máximo, contame que paso, en orden, y por que las tres correcciones de la variable no cambiaron nada. Tienes que sostener cada afirmación en una línea concreta de la evidencia. Hay dos problemas independientes en juego: no los mezcles.

- 1- El usuario intentó ingresar por www, disparando un bloqueo de CORS
- 2- La API no autorizaba ese origen ya que no incluye el www (<https://.api.tup2026>)
- 3- Cuando se intentaron aplicar los cambios nunca surgieron porque nunca se desplegó el contenedor
- 4- Por lo tanto, el proceso en ejecución continuó leyendo los valores cargados al inicio sin tomar las modificaciones
- 5- En paralelo e independiente el certificado de docs no emite ("certificado pendiente hace 40 minutos")
- 6- La causa de esto es porque el puerto HTTP-01 Let's Encrypt está bloqueado

5.2 Las tres contradicciones aparentes

6 puntos (2 cada una)

La evidencia parece contradecirse tres veces. Explica cada caso: (i) el OPTIONS figura con código 200 y aun así la operación falla, y del POST no hay rastro en la pestaña Red; (ii) el curl a la misma URL, con el mismo cuerpo, devuelve el resultado sin ningún error; (iii) los tres dominios resuelven a la misma dirección IP y sin embargo dos tienen certificado y el tercero no.

- (i) El servidor responde joya, (responde con 200 – OK) a la petición OPTIONS, lo que pasa es que al omitir la cabecera de Acces-Control-Allow-Origin (Para ese origen) ("No 'Acces-Control-Allow-Origin' is present". . .). Al no recibir dicha autorización, el navegador aborta el flujo y nunca emite ninguna petición real POST
- (ii) CORS como la teoría bien dice, es una política de seguridad impuesta por el navegador para proteger a los usuarios. Las herramientas de terminal como curl, por ejemplo, no son navegadores ni aplican restricciones de mismo origen, por lo que envían el método POST directo y consumen la respuesta sin intermediación del cliente
- (iii) Aunque el DNS resuelva correctamente por el comodín, la emisión automática vía ACME va a seguir requiriendo superar el desafío de HTTP-01 conectándose al puerto 80. La evidencia que se muestra, es que el puerto 80 se cerró o filtro después de haber emitido los certificados de la api y calculadora, lo que impidió a Lets Encrypt validar y emitir el nuevo certificado para docs

5.3 Los principios violados

4 puntos

Nombra los principios del capítulo que el grupo violó y en qué decisión concreta. Uno de ellos ya lo viste en la Clase 2 y aparece acá aplicado al panel: decime cual y como se traduce.

Ciclo de vida de las variables de entorno: Se violó el principio operativo donde se establece que las variables de entorno se leen al arrancar el proceso y modificarlas no tiene ningún efecto hasta redesplegar(Deploy)

- Decisión Concreta -> Cuando cambiaron la variable tres veces esperando que los cambios se aplicarán al instante sin presionar el botón de Deploy Permanencia en el puerto 80 para el Protocolo ACME: Se violó el requisito donde el puerto 80 debe permanecer abierto para siempre para validar los desafíos HTTP-01 de Let's Encrypt
- Decisión Concreta: Filtraron el puerto 80 en el firewall del servidor tras haber obtenido los primeros certificados, lo que provocó que el subdominio docs quede

pendiente indefinidamente Comparación literal de orígenes en la Política de Mismo Origen: Se violó el principio de que la comparación de orígenes es literal sobre esquema, anfitrión y puerto, donde un subdominio “www” es un anfitrión distinto al dominio raíz

- Decisión Concreta: accedieron desde https://www.calculadora... Habiendo declarado en la API únicamente la segunda mencionada, https://www.calculadora...

Principio mínimo de privilegio: Cuando se conecta Github con Easypanel, se debe autorizar el acceso únicamente a los repositorios estrictamente necesarios del proyecto y no seleccionar todos. Si el servidor o el panel resultan comprometidos, el atacante sólo podrá acceder a los repositorios concedidos y no a toda la cuenta.

5.4 Las cuatro correcciones, en orden

4 puntos

Escribí las cuatro medidas que aplicamos, en el orden en que las aplicamos. Justifica el orden, no solo el contenido: dos de las cuatro no se pueden intercambiar, y una es más urgente de lo que parece.

1- Reabrir el puerto 80 en el firewall del servidor (ufw allow 80/tcp) esto ayuda a desbloquear el tráfico TCP entrante para que Lets Encrypt pueda completar el desafío HTTP-01

- Es la acción más urgente de todas porque lleva 40 minutazos de fallos seguidos. Si se demoran mas Lets Encrypt les va a aplicar un límite de tasa que bloqueara la emisión del certificado para docs.tup2026.com por días

2-Actualizar la variable ORIGENES_PERMITIDOS en el servicio api, hay que declarar ambos orígenes separados por coma:

ORIGENES_PERMITIDOS=https://calculadora.tup2026.com,https://www.calculadora.tup2026.com

- Antes de reiniciar o redespargar el servicio esto tuvo que haber realizado. El contenedor solo va a cargar sus variables de entorno al inicializar el proceso, por lo que el nuevo valor tiene que quedar persistido en el panel antes de invocar el arranque de nuevo

3-Redespargar el servicio api (Deploy) para ejecutar un nuevo despliegue del servicio backend desde la interfaz del Easypanel

- Este paso y el anterior no se pueden invertir nunca, si se redespiega primero, se vuelve a levantar la configuración; si se modifica la variable y no se redespiega, el proceso va a seguir leyendo el valor viejo en memoria

4-Solicitar la reemisión del certificado o redespargar el servicio para que Traefik fuerce la solicitud del certificado TLS ante Lets Encrypt para el subdominio docs.tup2026.com

- Este paso es el final porque requiere como condición previa e indispensable que el puerto 80 ya se encuentre abierto y alcanzable desde internet, solicitarlo antes de abrir el puerto volvería a fallar

Parte 6 - El plan de publicacion de tu propio proyecto

10 puntos

Esta parte no es un ejercicio de laboratorio: es la ficha con la que vas a completar los formularios cuando tengas el panel delante. Retoma la tabla de imagenes de la Parte 6 del taller anterior, un nivel mas afuera: alla decidiste **que entra a cada imagen**; aca decidis **con que nombre sale a internet, por que puerto se lo alcanza y que valores tiene que conocer al arrancar**.

Servicio	Repositorio y Build path	Dominio público	Puerto interno y de dónde sale ese número	Variables de entorno: cual, quien la lee y cuando
Frontend	nombreRepositorio/calculadora-frontend	calculadora.lucasavila.com	Puerto 80 Sale del puerto estándar por defecto donde escucha el servidor web	API_URL=[https://api.lucasavila.com] (https://api.lucasavila.com) Quien la lee: El script docker-entrypoint.sh lo lee Cuando: Al arrancar el contenedor, para generar dinámicamente el archivo config.js consumido por el navegador
API	tu-usuario/calculadora-back end	api.lucasavila.com	Puerto 8000 Sale del puerto declarado en elDockerfile y utilizado por el servidor ASGI uvicorn	ORIGENES_PERMITIDOS=https://calculadora.lucasavila.com Quien la lee: el backend en FastAPI Cuando: Al inicializarla para configurar y validar orígenes autorizados

6.1 Las dos celdas que se miran entre sí

5 puntos

En tu tabla hay una dependencia cruzada: un dato de la fila del front end está escrito dentro de la fila de la API, y viceversa. Identifica, explica por qué existe, y deci que tenes que hacer -paso por paso- si mañana el grupo decide cambiar el dominio del frontend.

Frontend	API
Contiene el dominio de la API	Contieneel dominio del Frontend

¿Por qué existe? - Para separar en dos servicios independientes y el mecanismo de seguridad CORS. El navegador necesita conocer la URL pública de la API para poder dirigir las peticiones HTTP. A su vez, como la llamada ocurren en dos orígenes distintos, la API debe declarar ORIGENES_PERMITIDOS que autoriza a ese dominio frontend a leer sus respuestas

Si mañana nos aloamos y cambiamos todo esta sería la serie de pasos a seguir:

1-Actualizar el dominio en el servicio Frontend: Ir a Domains & Proxy del servicio web en Easypanel y cambiar el dominio asignado por el nuevo

2-Actualizar la variable de entorno en la API: Ir a Environment del servicio API y modificar ORIGENES_PERMITIDOS colocando el nuevo origen completo con esquema y sin barra al final

3-Guardar los cambios: Al final aparece un botoncito de "Save"

4-Redesplegar el servicio API: Presionar el botón de Deploy en el servicio y ya está, el contenedor reinicia su proceso, carga en memoria lo nuevo y listo

6.2 La línea que te vas a agradecer

5 puntos

Mira tu tabla y responde: si mañana alguien de tu grupo dice "pongamos ORIGENES_PERMITIDOS=* y listo, asi no perdemos más tiempo con esto y anda desde cualquier lado", que le decís? Escribí la respuesta técnica y la alternativa concreta que le ofreces, porque la necesidad es legítima.

Le digo que no toque si no sabe, porque usar un comodín (*) desactiva el mecanismo de seguridad del navegador y expone a los usuarios a ataques de falsificación de peticiones e inyección de scripts, permitiendo que cualquier sitio web malicioso ejecute peticiones a la API en segundo plano usando la sesión de navegador del usuario.

La alternativa concreta (sin explotar la seguridad): En lugar de abrir con un comodín (*), se mantiene una lista blanca explícita de orígenes declarados y separados por coma en la variable de entorno

- ¿Por qué resuelve el problema? Porque permite incorporar todos los entornos legítimos de desarrollo local junto con los subdominios de producción con y sin www

- ¿Por qué es seguro? Porque mantiene restringido el acceso únicamente a los entornos controlados por el grupo, evitando que cualquier dominio arbitrario de internet interactúe con la API en nombre de los clientes

Cierre - Para discutir en la mesa

sin puntaje

La conclusion del capitulo es incomoda: **la clase donde menos se programa es la clase donde mas cosas fallan**. Completar seis campos en dos formularios no deberia ser dificil, y sin embargo un numero mal puesto se manifiesta como un error del servidor, una barra de mas se manifiesta como un problema de red, y un puerto cerrado se manifiesta como un certificado que "queda pendiente". La razon es siempre la misma: **el formulario le habla a una pieza que no ves**, y el sintoma aparece donde esa pieza toca al mundo, no donde vos escribiste el valor.

Discutan cinco minutos, sin escribir: en todo lo que vieron hoy, cuantas veces el panel dijo algo que era cierto y aun asi enganaba? El servicio decia `running`. El certificado decia `pendiente`. El log decia `200 OK`. La pantalla mostraba la variable cargada. Ninguno mentia, y ninguno respondia la pregunta que importaba. Para cada uno de esos cuatro indicadores: que pregunta contesta realmente, y con que herramienta se contesta la que uno queria hacer?

Lo que se entrega

Las hojas del taller, con el nombre de los tres integrantes en la primera. Si la Parte 6 les quedo a medias, entreguenla igual como esta: esa ficha es lo que van a copiar en los formularios cuando tengan el panel delante, y la vamos a retomar con el proyecto andando. Lo que **no** se entrega en blanco es la justificacion: prefiero una decision discutible bien fundada que una correcta sin razon.

- Un único servidor con una sola IP puede alojar múltiples dominios gracias a la cabecera Host de HTTP/1.1 y a la extensión SNI de TLS en conexiones cifradas.
- Traefik actúa como proxy inverso en los puertos 80 y 443; descubre contenedores mediante Docker sin necesidad de editar archivos y enruta las peticiones según el dominio.
- Easypanel opera como una plataforma sobre Docker, Traefik y Let's Encrypt para traducir formularios a etiquetas, variables y órdenes de construcción.

Despliegue y Flujo de Contenedores

- El código se organiza en dos repositorios independientes con contexto de construcción en `/`.
- El repositorio remoto es la única fuente de verdad; Easypanel clona, lee el Dockerfile, compila la imagen y la ejecuta.
- Los puertos definidos en el panel corresponden a los puertos internos del contenedor (8000 para la API con Uvicorn y 80 para el Frontend con Nginx), no a puertos expuestos del VPS.

Certificados TLS y Protocolo ACME

- Let's Encrypt emite certificados automáticos de 90 días mediante el protocolo ACME y el desafío HTTP-01.
- La validación requiere obligatoriamente que el dominio resuelva hacia la IP del servidor y que el puerto 80 permanezca abierto permanentemente para las renovaciones.

Política de Mismo Origen y CORS

- Un origen es la combinación estricta de esquema, anfitrión y puerto.
- CORS es una política aplicada por el navegador (no por el servidor) para evitar que un origen lea datos de otro sin permiso explícito.
- Las peticiones complejas (como las que usan application/json) disparan una verificación previa; si el servidor no incluye al frontend en ORIGENES_PERMITIDOS, el navegador bloquea la llamada y el POST real no se emite.
- Clientes como curl o scripts de backend ignoran CORS por completo, ya que este mecanismo no funciona como control de acceso de la API.

Diagnóstico y Operación

- Error 404: Ningún enrutador de Traefik coincidió con el dominio solicitado.
- Error 502: El enrutador coincidió pero el servicio interno no respondió (puerto incorrecto o servicio escuchando únicamente en 127.0.0.1 en vez de 0.0.0.0).
- Modificar variables de entorno no surte efecto en el proceso hasta que se realiza un nuevo despliegue (Deploy).